

WHATSAPP WEB CLONE

Complete API & Real-Time Socket Documentation

Backend REST Endpoints + WebRTC & Socket.IO Signaling

DOCUMENT SUMMARY & SPECIFICATION

Project Name:	WhatsApp Full-Stack Application (Express + MySQL + Socket.IO + React)
Base REST URL:	http://localhost:3001/api (Proxy: /api)
Socket Endpoint:	ws://localhost:3001 (Namespace: /)
Authentication:	Phone Number + 6-Digit Country-Code Validated OTP
Key Features:	1-to-1 Chat, GPS Location Sharing, WebRTC Calling, Real-Time Ticks, Profile & Privacy, Block/Unblock, Message Edit & Edit History, Delete for Me/Everyone
Document Version:	2.0 (Updated September 2026)
Author / Maintainer:	DeepMind Agentic Engineering

TABLE OF CONTENTS

SECTION 1: Authentication & OTP REST APIs	3 Endpoints
SECTION 2: Contacts & Conversation REST APIs	4 Endpoints
SECTION 3: User Profile & Privacy REST APIs	2 Endpoints
SECTION 4: Block & Security REST APIs	4 Endpoints
SECTION 5: Real-Time Socket.IO Architecture & Events	12 Client & Server Events
SECTION 6: WebRTC Audio/Video Signaling Events	5 Signaling Handlers

SECTION 1: AUTHENTICATION & OTP REST APIS

Handles phone number country-code validation, 6-digit OTP generation and user session creation.

POST /api/auth/send-otp

API PURPOSE: Generates 6-digit OTP for phone number and saves in DB with 5-min expiration.

REQUEST BODY (JSON):

```
{
  "countryCode": "+91",
  "phone": "9876543210"
}
```

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "message": "OTP sent successfully to +919876543210 and saved in database",
  "data": {
    "countryCode": "+91", "phone": "9876543210", "fullPhone": "+919876543210",
    "expiresIn": "5 minutes", "otp": "481920"
  }
}
```

POST /api/auth/verify-otp

API PURPOSE: Verifies OTP against database and registers or logs in user, returning user session.

REQUEST BODY (JSON):

```
{
  "countryCode": "+91",
  "phone": "9876543210",
  "otp": "481920",
  "name": "Akash Rai"
}
```

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "message": "Login successful",
  "user": {
    "id": "+919876543210", "name": "Akash Rai", "countryCode": "+91",
    "phone": "9876543210", "fullPhone": "+919876543210",
    "about": "Available", "avatar": "???", "avatarPrivacy": "everyone"
  }
}
```

GET /api/auth/me/:id

API PURPOSE: Fetches registered user profile details by ID or full phone number.

PARAMETERS:

Path Param: id (string) - e.g. '+919876543210'

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "user": {
    "id": "+919876543210", "name": "Akash Rai", "fullPhone": "+919876543210",
    "about": "Hey there! I am using WhatsApp", "avatar": "???"
  }
}
```

SECTION 2: CONTACTS & CONVERSATION REST APIS

Start conversations, search users, retrieve inbox list, and load message history.

POST /api/chat/contacts

API PURPOSE: Adds new contact to address book / creates direct contact entry.

REQUEST BODY (JSON):

```
{
  "countryCode": "+91",
  "phone": "8382800528",
  "name": "Rahul Kumar"
}
```

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "message": "Contact added successfully",
  "contact": { "id": "+918382800528", "fullPhone": "+918382800528", "name": "Rahul Kumar" }
}
```

GET /api/chat/users/search

API PURPOSE: Searches users in DB by exact/partial phone number or name.

PARAMETERS:

Query Param: q (string) - search query e.g. '8382' or '+918382800528'

Query Param: currentUserId (string) - excludes self from results

RESPONSE (200 OK / JSON):

```
{
  "success": true, "count": 1,
  "users": [
    { "id": "+918382800528", "name": "Rahul Kumar", "fullPhone": "+918382800528", "avatar": "???" }
  ]
}
```

GET /api/chat/conversations/:userId

API PURPOSE: Retrieves all active conversations for the user's inbox list with last message snippet.

PARAMETERS:

Path Param: userId (string) - e.g. '+917991877018'

RESPONSE (200 OK / JSON):

```
{
  "success": true, "userId": "+917991877018",
  "conversations": [
    {
      "id": "+918382800528", "name": "Rahul Kumar", "fullPhone": "+918382800528",
      "lastMessage": "Hello brother", "lastMessageType": "text",
      "lastMessageStatus": "seen", "lastMessageAt": "2026-09-07T07:15:00.000Z"
    }
  ]
}
```

GET /api/chat/history

API PURPOSE: Retrieves full 1-to-1 conversation messages between two users (excluding deleted-for-me).

PARAMETERS:

Query Param: userId (string) - current user ID

Query Param: otherUserId (string) - recipient ID

RESPONSE (200 OK / JSON):

```
{
  "success": true, "userId": "+917991877018", "otherUserId": "+918382800528",
  "messages": [
    {
      "id": "101", "from": "+917991877018", "to": "+918382800528",
      "text": "Shared location", "type": "location", "status": "seen",
      "mediaUrl": null, "originalText": null, "editedAt": null
    }
  ]
}
```

SECTION 3: USER PROFILE & PRIVACY REST APIS

Manage user status bio, avatar image, and WhatsApp privacy visibility settings.

GET /api/chat/profile/:id

API PURPOSE: Gets user profile evaluated against viewer ID privacy rules and block status.

PARAMETERS:

Path Param: id (string) - user ID e.g. '+918382800528'

Query Param: viewerId (string) - viewer user ID to check privacy & block rules

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "user": {
    "id": "+918382800528", "name": "Rahul Kumar", "about": "Available",
    "avatar": "???", "avatarPrivacy": "everyone",
    "isBlockedByMe": false, "isBlockedByThem": false
  }
}
```

PUT /api/chat/profile/:id

API PURPOSE: Updates user name, about status, avatar picture, and avatar privacy ('everyone' | 'contacts' | 'nobody').

PARAMETERS:

Path Param: id (string) - user ID e.g. '+917991877018'

REQUEST BODY (JSON):

```
{
  "name": "Akash Rai",
  "about": "Busy at work ?",
  "avatar": "https://example.com/photo.jpg",
  "avatarPrivacy": "everyone"
}
```

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "message": "Profile updated successfully",
  "user": { "id": "+917991877018", "name": "Akash Rai", "about": "Busy at work ?" }
}
```

SECTION 4: BLOCK & SECURITY REST APIS

Block, unblock contacts, and inspect privacy enforcement status.

POST /api/chat/block

API PURPOSE: Blocks a contact. Hides profile picture, status bio, presence, and prevents calls & messages.

REQUEST BODY (JSON):

```
{
  "blockerId": "+917991877018",
  "blockedId": "+918382800528"
}
```

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "message": "User blocked successfully",
  "blockerId": "+917991877018", "blockedId": "+918382800528"
}
```

POST /api/chat/unblock

API PURPOSE: Unblocks a previously blocked contact and restores full chat & profile visibility.

REQUEST BODY (JSON):

```
{
  "blockerId": "+917991877018",
  "blockedId": "+918382800528"
}
```

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "message": "User unblocked successfully"
}
```

GET /api/chat/block-status

API PURPOSE: Checks bidirectional block status between two users.

PARAMETERS:

Query Param: userId (string) - user A

Query Param: otherUserId (string) - user B

RESPONSE (200 OK / JSON):

```
{
  "success": true,
  "isBlockedByMe": false,
  "isBlockedByThem": false
}
```

GET /api/chat/blocked/:userId

API PURPOSE: Retrieves list of all contact IDs blocked by this user.

PARAMETERS:

Path Param: userId (string) - e.g. '+917991877018'

RESPONSE (200 OK / JSON):

```
{
  "success": true, "userId": "+917991877018",
  "blockedUsers": [ "+918115454843" ]
}
```

SECTION 5: REAL-TIME SOCKET.IO ARCHITECTURE & EVENTS

Bi-directional WebSocket protocol events for real-time messaging, status ticks, presence, and deletion.

CLIENT -> SERVER `socket.emit('join')`

EVENT PURPOSE: Authenticates socket connection, joins user room, marks pending messages delivered, and broadcasts ONLINE status.

PAYLOAD DATA:

```
// Payload is user ID string
"+917991877018"
```

SERVER BROADCAST / ACK:

```
1. socket.emit("joined", "+917991877018")
2. io.emit("user_status", { userId: "+917991877018", isOnline: true })
3. socket.emit("online_users", [ "+917991877018", "+918382800528" ])
```

CLIENT -> SERVER `socket.emit('message')`

EVENT PURPOSE: Sends 1-to-1 message (Text, Image Photo, or ? GPS Location). Computes WhatsApp Ticks (sent/delivered/seen).

PAYLOAD DATA:

```
{
  "to": "+918382800528",
  "text": "{ \"latitude\":28.6139, \"longitude\":77.2090, \"title\": \"New Delhi\" }",
  "type": "location", // "text" | "image" | "location"
  "mediaUrl": null
}
```

SERVER BROADCAST / ACK:

```
1. io.to(to).emit("message_received", savedMessage)
2. socket.emit("message_saved", savedMessage)
```

CLIENT -> SERVER `socket.emit('loadConversation')`

EVENT PURPOSE: Opens conversation with recipient, marks received messages as SEEN (Double Blue Tick), and sends history.

PAYLOAD DATA:

```
"+918382800528"
```

SERVER BROADCAST / ACK:

```
1. io.to(otherUser).emit("messages_seen", { seenBy: userId })
2. socket.emit("conversation_history", [ messageObj1, messageObj2, ... ])
```

CLIENT -> SERVER `socket.emit('typing')`

EVENT PURPOSE: Emits real-time typing indicator to peer when user is typing in chat input.

PAYLOAD DATA:

```
{ "to": "+918382800528", "isTyping": true }
```

SERVER BROADCAST / ACK:

```
io.to(to).emit("typing", { from: "+917991877018", isTyping: true })
```

CLIENT -> SERVER `socket.emit('delete_message')`**EVENT PURPOSE:** Handles WhatsApp-style 'Delete for everyone' (global tombstone) vs 'Delete for me' (local hide).**PAYLOAD DATA:**

```
{
  "messageId": "105",
  "deleteFor": "everyone", // "everyone" | "me"
  "to": "+918382800528"
}
```

SERVER BROADCAST / ACK:

```
If everyone: io.to(to).emit("message_deleted", { messageId: "105", deleteFor: "everyone", message })
              socket.emit("message_deleted", { messageId: "105", deleteFor: "everyone", message })
If me:       socket.emit("message_deleted", { messageId: "105", deleteFor: "me" })
```

CLIENT -> SERVER `socket.emit('edit_message')`**EVENT PURPOSE:** Updates message text in DB, preserves original_text, and syncs Edited badge & text to peer instantly.**PAYLOAD DATA:**

```
{
  "messageId": "105",
  "text": "Updated new message text",
  "to": "+918382800528"
}
```

SERVER BROADCAST / ACK:

```
io.to(to).emit("message_edited", updatedMessageObj)
socket.emit("message_edited", updatedMessageObj)
```

SECTION 6: WEBRTC AUDIO/VIDEO SIGNALING EVENTS

Peer-to-peer WebRTC signaling protocol for WhatsApp Voice & Video calls.

CLIENT -> SERVER `socket.emit('call_user')`**EVENT PURPOSE:** Initiates voice/video call and sends WebRTC SDP Offer with caller details and ringtone trigger.**PAYLOAD DATA:**

```
{
  "to": "+918382800528",
  "callerName": "Akash Rai",
  "callerAvatar": "???",
  "callType": "video", // "voice" | "video"
  "offer": { "type": "offer", "sdp": "..." }
}
```

SERVER BROADCAST / ACK:

```
io.to(to).emit("incoming_call", { from, callerName, callerAvatar, callType, offer })
```

CLIENT -> SERVER `socket.emit('answer_call')`**EVENT PURPOSE:** Accepts incoming call and returns WebRTC SDP Answer to establish peer media stream.**PAYLOAD DATA:**

```
{
  "to": "+917991877018",
  "answer": { "type": "answer", "sdp": "..." }
}
```

SERVER BROADCAST / ACK:

```
io.to(to).emit("call_accepted", { from: callerId, answer })
```

CLIENT -> SERVER `socket.emit('ice_candidate')`**EVENT PURPOSE:** Exchanges interactive connectivity establishment (ICE) network candidates between caller and callee.**PAYLOAD DATA:**

```
{
  "to": "+918382800528",
  "candidate": { "candidate": "...", "sdpMid": "0", "sdpMLineIndex": 0 }
}
```

SERVER BROADCAST / ACK:

```
io.to(to).emit("ice_candidate", { from, candidate })
```

CLIENT -> SERVER `socket.emit('end_call / reject_call')`**EVENT PURPOSE:** Terminates active call or declines incoming call and stops audio ringtones.**PAYLOAD DATA:**

```
{ "to": "+918382800528", "reason": "Declined" }
```

SERVER BROADCAST / ACK:

```
io.to(to).emit("call_ended", { from }) / io.to(to).emit("call_rejected", { from, reason })
```